

MAR ATHANASIOS COLLEGE FOR ADVANCED STUDIES, TIRUVALLA

(MACFAST)

(Affiliated to Mahatma Gandhi University, Kottayam)



MACFAST (AUTONOMOUS)

MCACP106 DATABASE TECHNOLOGY LAB (MYSQL & MONGODB)

Department of Computer Applications

Mar Athanasios College for Advanced Studies, Tiruvalla (MACFAST)

Kerala, India, Pin-689101

www.macfast.org

www.macfast.ac.in

MAR ATHANASIOS COLLEGE FOR ADVANCED STUDIES, TIRUVALLA

(MACFAST)

(Affiliated to Mahatma Gandhi University, Kottayam)



MACFAST (AUTONOMOUS)

MCACP106 DATABASE TECHNOLOGY LAB (MYSQL & MONGODB)

Name : PRINTO MATHEW

Register No : 253242210258

Semester : 1

Subject : Database Technology Lab (Mysql & Mongodb)

*Certified that this is a Bonafide record of the practical work done by
Mr. PRINTO MATHEW Reg.No: 253242210258 in the Department of Computer Applications
during 2025-2027 at Mar Athanasios College for Advanced Studies, Tiruvalla ((MACFAST)
Autonomous).*

Faculty in Charge

Head of the Department

Examiners

1.....

2.....

INDEX

SL.NO	DATE	NAME OF PROGRAM	PAGE NO
1	12/09/2025	Creating tables and inserting values	4
2	16/09/2025	Modifying the table structure	7
3	19/09/2025	Insertion and updation of records	10
4	07/10/2025	Demonstrating data retrieval and joining multiple tables: a) Basic Queries b) Demonstrating the use of UNION, INTERSECT, EXCEPT/MINUS. c) Nested Queries d) Correlated Nested Query e) Aggregate Operators f) Set Comparison Operators	16
5	28/10/2025	Demonstrating GROUP BY & HAVING Clause	27
6	05/11/2025	Number, String & Date Functions	29
7	19/11/2025	VIEWS	37
8	27/11/2025	Mongodb	39

I. CREATING TABLES AND INSERTING VALUES

1. Create table customer with following fields:

COLUMN NAME	TYPE	REMARKS
cid	Integer	
fname	varchar(10)	
lname	varchar(50)	
age	Integer	
gender	varchar(5)	Should allow only F or M
state	varchar(30)	
pincode	Integer	

OUTPUT:

```
SQL> CREATE table customer(cid int,fname varchar(10),lname varchar(50),age int,gender varchar(5) CHECK(gender in('M','F')),  
state varchar(30),pincode int);
```

Table created.

```
SQL> desc customer;
```

Name	Null?	Type
CID		NUMBER(38)
FNAME		VARCHAR2(10)
LNAME		VARCHAR2(50)
AGE		NUMBER(38)
GENDER		VARCHAR2(5)
STATE		VARCHAR2(30)
PINCODE		NUMBER(38)

2. Create table sailors with following fields:

COLUMN NAME	TYPE	REMARKS
sid	Integer	PRIMARY KEY
sname	varchar(20)	
rating	Integer	
age	Integer	

OUTPUT:

```
SQL> CREATE table sailor(sid int PRIMARY KEY,sname varchar(20),rating int,age int);
Table created.

SQL> desc sailor;
+-----+-----+-----+
Name                               Null?   Type
+-----+-----+-----+
SID                                NOT NULL NUMBER(38)
SNAME                             VARCHAR2(20)
RATING                             NUMBER(38)
AGE                                NUMBER(38)
```

3. Create table boats with following fields:

COLUMN NAME	TYPE	REMARKS
bid	Integer	PRIMARY KEY
bname	varchar(20)	
color	varchar(20)	

OUTPUT:

```
SQL> CREATE table boats(bid int PRIMARY KEY,bname varchar(20),color varchar(20));
Table created.

SQL> desc boats;
+-----+-----+-----+
Name                               Null?   Type
+-----+-----+-----+
BID                                NOT NULL NUMBER(38)
BNAME                             VARCHAR2(20)
COLOR                             VARCHAR2(20)
```

4. Create table reserves with following fields:

COLUMN NAME	TYPE	REMARKS
sid	Integer	PK,FK
bid	Integer	PK,FK
day	date	

OUTPUT:

```
SQL> CREATE table reserves(sid int,bid int,day date,PRIMARY KEY(sid,bid),
    FOREIGN KEY (sid) REFERENCES sailor(sid),FOREIGN KEY (bid) REFERENCES boats(bid));
```

Table created.

```
SQL> desc reserves;
```

Name	Null?	Type
SID	NOT NULL	NUMBER(38)
BID	NOT NULL	NUMBER(38)
DAY		DATE

5. Display the description of customer table.

OUTPUT:

```
SQL> desc customer;
```

Name	Null?	Type
CID		NUMBER(38)
FNAME		VARCHAR2(10)
LNAME		VARCHAR2(50)
AGE		NUMBER(38)
GENDER		VARCHAR2(5)
STATE		VARCHAR2(30)
PINCODE		NUMBER(38)

II. MODIFYING THE TABLE STRUCTURE

1. Rename the table sailor to sailors.

OUTPUT:

```
SQL> ALTER table sailor RENAME TO sailors;
```

```
Table altered.
```

```
SQL> desc sailors;
```

Name	Null?	Type
SID	NOT NULL	NUMBER(38)
SNAME		VARCHAR2(20)
RATING		NUMBER(38)
AGE		NUMBER(38)

2. Rename the field pincode in customer table to pin.

OUTPUT:

```
SQL> ALTER table customer RENAME COLUMN pincode TO pin;
```

```
Table altered.
```

```
SQL> desc customer;
```

Name	Null?	Type
CID		NUMBER(38)
FNAME		VARCHAR2(10)
LNAME		VARCHAR2(50)
AGE		NUMBER(38)
GENDER		VARCHAR2(5)
STATE		VARCHAR2(30)
PIN		NUMBER(38)

3. Add a new column DOB to customer table.

OUTPUT:

```
SQL> ALTER table customer ADD DOB date;
```

Table altered.

```
SQL> desc customer;
```

Name	Null?	Type
-----	-----	-----
CID		NUMBER(38)
FNAME		VARCHAR2(10)
LNAME		VARCHAR2(50)
AGE		NUMBER(38)
GENDER		VARCHAR2(5)
STATE		VARCHAR2(30)
PIN		NUMBER(38)
DOB		DATE

4. Change the size of fname column to varchar(20)

OUTPUT:

```
SQL> ALTER table customer MODIFY fname varchar(20);
```

Table altered.

```
SQL> desc customer;
```

Name	Null?	Type
-----	-----	-----
CID		NUMBER(38)
FNAME		VARCHAR2(20)
LNAME		VARCHAR2(50)
AGE		NUMBER(38)
GENDER		VARCHAR2(5)
STATE		VARCHAR2(30)
PIN		NUMBER(38)
DOB		DATE

5. Add a constraint to the DOB field of customer table which allows only dates greater than '01-JAN-1980'.

OUTPUT:

```
SQL> ALTER table customer ADD CONSTRAINT chk_dob CHECK(dob > '01-JAN-1980');
```

```
Table altered.
```

```
SQL> desc customer;
```

Name	Null?	Type
-----	-----	-----
CID		NUMBER(38)
FNAME		VARCHAR2(20)
LNAME		VARCHAR2(50)
AGE		NUMBER(38)
GENDER		VARCHAR2(5)
STATE		VARCHAR2(30)
PIN		NUMBER(38)
DOB		DATE

III. INSERTION AND UPDATION OF RECORDS

1. Insert records into customer table.

OUTPUT:

```
SQL> INSERT INTO customer VALUES(100,'ANNA','TOM',21,'M','KERALA',684521,'05-MAR-2024');
1 row created.

SQL> INSERT INTO customer VALUES(101,'DEVI','KUMAR',45,'F','MADHYA PRADESH',697820,'21-FEB-1980');
1 row created.

SQL> INSERT INTO customer VALUES(102,'RENU','PRADEEP',31,'F','KERALA',640963,'15-DEC-1996');
1 row created.

SQL> INSERT INTO customer VALUES(103,'AKHIL','SURESH',40,'F','KERALA',647178,'02-FEB-1985');
1 row created.

SQL> INSERT INTO customer VALUES(104,'INAO','KABUI',27,'M','MANIPUR',636213,'12-JUL-1998');
1 row created.

SQL> INSERT INTO customer VALUES(105,'SUBASH','PATEL',36,'M','GUJARAT',869035,'09-OCT-1991');
1 row created.

SQL> INSERT INTO customer VALUES(107,'YUVARAJ','SINGH',42,'M','PUNJAB',701335,'19-NOV-1987');
1 row created.

SQL> INSERT INTO customer VALUES(108,'ANJALI','KAMAL',39,'F','DELHI',961839,'27-NOV-1989');
1 row created.
```

2. List all the contents of customer table.

OUTPUT:

```
SQL> SELECT * FROM customer;
```

CID	FNAME	LNAME	AGE	GENDE	STATE	PIN	DOB
100	ANNA	TOM	21	M	KERALA	684521	05-MAR-24
101	DEVI	KUMAR	45	F	MADHYA PRADESH	697820	21-FEB-80
102	RENU	PRADEEP	31	F	KERALA	640963	15-DEC-96
103	AKHIL	SURESH	40	F	KERALA	647178	02-FEB-85
104	INAO	KABUI	27	M	MANIPUR	636213	12-JUL-98
105	SUBASH	PATEL	36	M	GUJARAT	869035	09-OCT-91
107	YUVARAJ	SINGH	42	M	PUNJAB	701335	19-NOV-87
108	ANJALI	KAMAL	39	F	DELHI	961839	27-NOV-89
110	HIMA	PREM	20	F	UTTAR PRADESH	961796	16-AUG-05
109	ABDUL	KALAM	25	M	TAMIL NADU		10-SEP-00

```
10 rows selected.
```

3. Retrieve the full names of all customers, the output should be titled “CUSTOMER LIST”.

OUTPUT:

```
SQL> SELECT fname || ' ' || lname AS "CUSTOMER LIST" FROM customer;

CUSTOMER LIST
-----
ANNA TOM
DEVI KUMAR
RENU PRADEEP
AKHIL SURESH
INAO KABUI
SUBASH PATEL
YUVARAJ SINGH
ANJALI KAMAL
ABDUL KALAM
HIMA PREM

10 rows selected.
```

4. Retrieve First name and last name of customer from Kerala.

OUTPUT:

```
SQL> SELECT fname || ' ' || lname AS "CUSTOMER'S FROM KERALA" FROM customer WHERE state='KERALA';

CUSTOMER'S FROM KERALA
-----
ANNA TOM
RENU PRADEEP
AKHIL SURESH
```

5. Retrieve the names of customers for whom pincode is missing.

OUTPUT:

```
SQL> SELECT fname || ' ' || lname AS "PINCODE MISSING CUSTOMER'S" FROM customer WHERE pin IS NULL;

PINCODE MISSING CUSTOMER'S
-----
ABDUL KALAM
```

6. Retrieve the name of state from which customers are there.

OUTPUT:

```
SQL> SELECT DISTINCT state FROM customer;

STATE
-----
PUNJAB
DELHI
TAMIL NADU
UTTAR PRADESH
MANIPUR
KERALA
MADHYA PRADESH
GUJARAT

8 rows selected.
```

7. Retrieve the name and corresponding state of north Indian Customers.

OUTPUT:

```
SQL> SELECT fname || ' ' || lname AS "CUSTOMER NAME",state FROM customer
2 WHERE state IN('UTTAR PRADESH','HIMACHAL PRADESH','JAMMU AND KASHMIR',
3 'PUNJAB','UTTARAKHAND','HARYANA','DELHI','RAJASTHAN');

CUSTOMER NAME                                STATE
-----
YUVARAJ SINGH                                PUNJAB
ANJALI KAMAL                                 DELHI
HIMA PREM                                    UTTAR PRADESH
```

8. Select fname & lname ,state as customer list from customer where state='Manipur' or state='Gujarat'.

OUTPUT:

```
SQL> SELECT fname || ' ' || lname AS "CUSTOMER LIST",state FROM customer
2 WHERE state = 'GUJARAT' OR state= 'MANIPUR';

CUSTOMER LIST                                STATE
-----
INAO KABUI                                   MANIPUR
SUBASH PATEL                                 GUJARAT
```

9. Retrieve customer information in decending order of state.

OUTPUT:

```
SQL> SELECT * FROM customer ORDER BY state DESC;
```

CID	FNAME	LNAME	AGE	GENDE	STATE	PIN	DOB
110	HIMA	PREM	20	F	UTTAR PRADESH	961796	16-AUG-05
109	ABDUL	KALAM	45	M	TAMIL NADU		01-SEP-80
107	YUVARAJ	SINGH	42	M	PUNJAB	701335	19-NOV-87
104	INAO	KABUI	27	M	MANIPUR	636213	12-JUL-98
101	DEVI	KUMAR	45	F	MADHYA PRADESH	697820	21-FEB-80
102	RENU	PRADEEP	31	F	KERALA	640963	15-DEC-96
100	ANNA	TOM	21	M	KERALA	684521	05-MAR-24
103	AKHIL	SURESH	40	F	KERALA	647178	02-FEB-85
105	SUBASH	PATEL	36	M	GUJARAT	869035	09-OCT-91
108	ANJALI	KAMAL	39	F	DELHI	961839	27-NOV-89

10 rows selected.

10. Retrieve customer information in alphabetical order of there names.

OUTPUT:

```
SQL> SELECT * FROM customer ORDER BY fname,lname;
```

CID	FNAME	LNAME	AGE	GENDE	STATE	PIN	DOB
109	ABDUL	KALAM	45	M	TAMIL NADU		01-SEP-80
103	AKHIL	SURESH	40	F	KERALA	647178	02-FEB-85
108	ANJALI	KAMAL	39	F	DELHI	961839	27-NOV-89
100	ANNA	TOM	21	M	KERALA	684521	05-MAR-24
101	DEVI	KUMAR	45	F	MADHYA PRADESH	697820	21-FEB-80
110	HIMA	PREM	20	F	UTTAR PRADESH	961796	16-AUG-05
104	INAO	KABUI	27	M	MANIPUR	636213	12-JUL-98
102	RENU	PRADEEP	31	F	KERALA	908876	15-DEC-96
111	SREEDEVI	RAJ	19	F	telangana	723104	19-JUN-06
105	SUBASH	PATEL	36	M	GUJARAT	869035	09-OCT-91
107	YUVARAJ	SINGH	42	M	PUNJAB	701335	19-NOV-87

11 rows selected.

11. Retrieve the names of customers from kerala whose pincode information is available.

OUTPUT:

```
SQL> SELECT fname || ' ' || lname AS "CUSTOMERS FROM KERALA" FROM customer
2 WHERE pin IS NOT NULL AND state='KERALA';
```

CUSTOMERS FROM KERALA

ANNA TOM
RENU PRADEEP
AKHIL SURESH

12. Select the names of all customers whose firstname contains the word 'devi'.

OUTPUT:

```
SQL> SELECT fname || ' ' || lname AS "CUSTOMER LIST" FROM customer WHERE fname LIKE '%DEVI%';
```

CUSTOMER LIST
DEVI KUMAR
SREEDEVI RAJ

13. Select the names of all customers whose names begin with 's' or 'r'.

OUTPUT:

```
SQL> SELECT fname|| ' ' || lname AS "CUSTOMER LIST" FROM customer WHERE fname LIKE 'S%' OR fname LIKE 'R%' ;
```

CUSTOMER LIST
RENU PRADEEP
SUBASH PATEL
SREEDEVI RAJ

14. Retrieve the names and age of all customers having cid greater than 103 and below 106.

OUTPUT:

```
SQL> SELECT fname || ' ' || lname AS "NAME", age FROM customer WHERE cid >103 AND cid <106;
```

NAME	AGE
INAO KABUI	27
SUBASH PATEL	36

15. Set the pincode as '908876' for 'renu'.

OUTPUT:

```
SQL> UPDATE customer SET pin=908876 WHERE fname='RENU';
```

1 row updated.

16. Retrieve the names of customers who are born after 01-JAN-1990.

OUTPUT:

```
SQL> SELECT fname || ' ' || lname AS "CUSTOMER LIST" FROM customer WHERE DOB > '01-JAN-1990';  
CUSTOMER LIST  
-----  
ANNA TOM  
RENU PRADEEP  
INAO KABUI  
SUBASH PATEL  
HIMA PREM
```

IV. DEMONSTRATING DATA RETRIEVAL AND JOINING MULTIPLE TABLES

1. List the contents of sailors table.

OUTPUT:

```
SQL> SELECT * FROM sailors;
```

SID	SNAME	RATING	AGE
1	RAHUL THOMAS	18	30
2	ZAHEER KHAN	13	23
3	VIRAT KOHLI	16	45
4	ROHIT SHARMA	20	48
5	KAPHIL DEV	24	78
6	PUSHPA RAJ	21	38
7	RAJU DHOLAKPUR	20	31
8	TANJIRO KAMADO	25	39

8 rows selected.

2. List the contents of boats table.

OUTPUT:

```
SQL> SELECT * FROM boats;
```

BID	BNAME	COLOR
200	MARCO	RED
201	RANNI	BLUE
202	ST.JOSEPH	GREEN
203	JKD BOATS	RED
204	TAJ	WHITE
205	STALIN	GREEN

6 rows selected.

3. List the contents of Reserves table.

OUTPUT:

```
SQL> SELECT * FROM reserves;
```

SID	BID	DAY
1	205	07-JUN-24
1	202	08-AUG-21
2	202	12-DEC-25
9	200	11-DEC-25
4	203	05-NOV-25

a. BASIC QUERIES

1. Find the names and ages of all sailors.

OUTPUT:

```
SQL> SELECT sname,age FROM sailors;
```

SNAME	AGE
RAHUL THOMAS	30
ZAHEER KHAN	23
VIRAT KOHLI	45
ROHIT SHARMA	48
KAPHIL DEV	78
PUSHPA RAJ	38
RAJU DHOLAKPUR	31
TANJIRO KAMADO	39
TOM	39

9 rows selected.

2. Retrieve the names and incremented ratings of all sailors(incremented rating=rating+5). The output should be headlined as 'new rating'.

OUTPUT:

```
SQL> SELECT sname, (rating + 5) AS "NEW RATING" FROM sailors;
```

SNAME	NEW RATING
RAHUL THOMAS	23
ZAHEER KHAN	18
VIRAT KOHLI	21
ROHIT SHARMA	25
KAPHIL DEV	29
PUSHPA RAJ	26
RAJU DHOLAKPUR	25
TANJIRO KAMADO	30
TOM	25

9 rows selected.

3. Find the names of sailors who have reserved boat 202.

OUTPUT:

```
SQL> SELECT sname FROM sailors s JOIN reserves r on s.sid = r.sid WHERE r.bid =202;

SNAME
-----
RAHUL THOMAS
ZAHEER KHAN
```

4. Find the names of sailors who reserved a red boat.

OUTPUT:

```
SQL> SELECT sname FROM sailors s JOIN reserves r on s.sid = r.sid
  2  JOIN boats b on r.bid = b.bid
  3  WHERE color = 'RED';

SNAME
-----
TOM
ROHIT SHARMA
```

5. Find the colors of boats reserved by tom.

OUTPUT:

```
SQL> SELECT color FROM boats b JOIN reserves r
  2  on r.bid = b.bid JOIN sailors s
  3  on s.sid = r.sid
  4  WHERE s.sname = 'TOM';

COLOR
-----
RED
```

6. Find the names of sailors who have reserved at least one boat.

OUTPUT:

```
SQL> SELECT DISTINCT sname FROM sailors S JOIN  
2 reserves R on r.sid = s.sid;
```

```
SNAME  
-----  
RAHUL THOMAS  
TOM  
ZAHEER KHAN  
ROHIT SHARMA
```

7. Compute the names and incremented ratings of all sailors who have sailed two different boats on the same day (incremented rating=rating+1). The incremented rating should appear in the output as 'newrating'.

OUTPUT:

```
SQL> SELECT s.sname, s.rating + 1 AS "New Rating"  
2 FROM sailors s  
3 WHERE s.sid IN (  
4 SELECT sid FROM reserves  
5 GROUP BY sid, day  
6 HAVING COUNT(bid) >= 2  
7 );
```

```
SNAME                New Rating  
-----  
RAHUL THOMAS                19
```

8. Find the names of sailors who have reserved a red or a green boat.

OUTPUT:

```
SQL> SELECT DISTINCT sname FROM sailors s
      2 JOIN reserves r on s.sid = r.sid
      3 JOIN boats b on r.bid = b.bid
      4 WHERE b.color IN ('RED','GREEN');
```

```
SNAME
-----
RAHUL THOMAS
TOM
ZAHEER KHAN
ROHIT SHARMA
```

b. DEMONSTRATING THE USE OF UNION , INTERSECT, EXCEPT / MINUS

1. Find the names of all sailors who have reserved a red or a green boat.

OUTPUT:

```
SQL> (SELECT sname
      2 FROM sailors s
      3 JOIN reserves r ON s.sid = r.sid
      4 JOIN boats b ON r.bid = b.bid
      5 WHERE b.color = 'RED')
      6 UNION
      7 (SELECT sname
      8 FROM sailors s
      9 JOIN reserves r on s.sid = r.sid
     10 JOIN boats b ON r.bid = b.bid
     11 WHERE b.color = 'GREEN');
```

```
SNAME
-----
RAHUL THOMAS
ROHIT SHARMA
TOM
ZAHEER KHAN
```

2. Find the names of all sailors who have reserved both red and green boats.

OUTPUT:

```
SQL> (SELECT sname
  2   FROM sailors s
  3   JOIN reserves r on s.sid = r.sid
  4   JOIN boats b ON r.bid = b.bid
  5   WHERE b.color = 'RED')
  6 INTERSECT
  7 (SELECT sname
  8   FROM sailors s
  9   JOIN reserves r on s.sid = r.sid
 10   JOIN boats b ON r.bid = b.bid
 11   WHERE b.color = 'GREEN');
```

SNAME

RAHUL THOMAS

3. Find the names of all sailors who have reserved a red boat but not a green boat.

OUTPUT:

```
SQL> (SELECT sname
  2   FROM sailors s
  3   JOIN reserves r on s.sid = r.sid
  4   JOIN boats b on r.bid = b.bid
  5   WHERE b.color = 'RED')
  6 MINUS
  7 (SELECT sname
  8   FROM sailors s
  9   JOIN reserves r on s.sid = r.sid
 10   JOIN boats b on r.bid = b.bid
 11   WHERE b.color = 'GREEN');
```

SNAME

ROHIT SHARMA

TOM

c. NESTED QUERIES

1. Find the names of sailors who have reserved a boat

OUTPUT:

```
SQL> SELECT sname FROM sailors WHERE sid IN (SELECT sid FROM reserves);

SNAME
-----
RAHUL THOMAS
ZAHEER KHAN
ROHIT SHARMA
TOM
```

2. Display names of sailors who have reserved a red boat

OUTPUT:

```
SQL> SELECT sname FROM sailors WHERE sid IN
  2  (SELECT r.sid FROM reserves r join boats b ON r.bid = b.bid
  3   WHERE b.color = 'RED');

SNAME
-----
RAHUL THOMAS
ROHIT SHARMA
TOM
```

3. Display sid and names of sailors who have not reserved a green boat.

OUTPUT:

```
SQL> SELECT sid,sname FROM sailors WHERE sid NOT IN
  2  (SELECT r.sid FROM reserves r JOIN boats b on r.bid = b.bid
  3   WHERE b.color = 'GREEN');

  SID SNAME
-----
    5 KAPHIL DEV
    8 TANJIRO KAMADO
    3 VIRAT KOHLI
    6 PUSHPA RAJ
    7 RAJU DHOLAKPUR
    4 ROHIT SHARMA
    9 TOM

7 rows selected.
```

d. CORRELATED NESTED QUERY

1. Find the names of all sailors who have reserved atleast one boat.

OUTPUT:

```
SQL> SELECT sname FROM sailors s WHERE EXISTS
      2  (SELECT 1 FROM reserves r WHERE r.sid = s.sid);

SNAME
-----
RAHUL THOMAS
ZAHEER KHAN
ROHIT SHARMA
TOM
```

2. Find the names of all sailors who have reserved boat number 201

OUTPUT:

```
SQL> INSERT INTO reserves VALUES(9,201,'10-OCT-2025');

1 row created.

SQL> SELECT sname FROM sailors s
      2  WHERE EXISTS (
      3      SELECT 1 FROM reserves r
      4      WHERE r.sid = s.sid AND r.bid = 201
      5  );

SNAME
-----
TOM
```

e. AGGREGATE OPERATORS

1. Find the average rating of all sailors

OUTPUT:

```
SQL> SELECT avg(rating) FROM sailors;

AVG(RATING)
-----
19.6666667
```

2. Retrieve the total number of sailors.

OUTPUT:

```
SQL> SELECT count(*) FROM sailors;

COUNT(*)
-----
9
```

3. Find the average age of all sailors with a rating of 20.

OUTPUT:

```
SQL> SELECT avg(age) FROM sailors WHERE rating = 20;

AVG(AGE)
-----
39.3333333
```

4. Find the names and age of oldest sailors.

OUTPUT:

```
SQL> SELECT sname AS "NAME", age AS "AGE" FROM sailors
2  WHERE age = (SELECT MAX(age) FROM sailors);
```

```
NAME                                AGE
-----
STEVE SMITH                        80
```


5. Find the names of sailors who are older than the oldest sailor with rating of 24.

OUTPUT:

```
SQL> SELECT sname FROM sailors
      2  WHERE age > (SELECT MAX(age) FROM sailors WHERE rating =24);

SNAME
-----
STEVE SMITH
```

f. SET COMPARISON OPERATORS

1. Find the sailors whose rating is better than some sailor called tom.

OUTPUT:

```
SQL> SELECT sname
      2  FROM sailors
      3  WHERE rating > ANY
      4  (SELECT rating FROM sailors WHERE sname='TOM');

SNAME
-----
TANJIRO KAMADO
JOHN SNOW
JOHN WICK
KAPHIL DEV
PUSHPA RAJ
STEVE SMITH

6 rows selected.
```

2. Find the sailors whose rating is better than every sailor called Rion.

OUTPUT:

```
SQL> SELECT sname FROM sailors WHERE rating > ALL
2  (SELECT rating FROM sailors
3   WHERE sname='RION');
```

SNAME

ROHIT SHARMA
KAPHIL DEV
PUSHPA RAJ
RAJU DHOLAKPUR
TANJIRO KAMADO
TOM
JOHN WICK
JOHN SNOW
STEVE SMITH

9 rows selected.

3. Find the sailors with highest rating.

OUTPUT:

```
SQL> SELECT sname,rating FROM sailors
2  WHERE rating = (
3   SELECT MAX(rating) FROM sailors);
```

SNAME

RATING

TANJIRO KAMADO 25

V. DEMONSTRATING GROUP BY AND HAVING CLAUSE

1. Retrieve the number of reservations for each boat

OUTPUT:

```
SQL> SELECT bid,COUNT(*) AS Total_Reservation
2  FROM reserves
3  GROUP BY bid;
```

BID	TOTAL_RESERVATION
205	1
201	1
200	1
202	2
203	2

2. Find the age of the youngest sailor for each rating level

OUTPUT:

```
SQL> SELECT rating,MIN(age) AS YOUNGEST FROM sailors GROUP BY rating;
```

RATING	YOUNGEST
25	39
13	23
20	31
21	38
24	68
18	30
16	45
19	34

3. Find the age of the youngest sailor for each rating level with atleast 2 sailors with that rating

OUTPUT:

```
SQL> SELECT rating,MIN(age) FROM sailors GROUP BY rating HAVING count(*) >= 2;
```

RATING	MIN(AGE)
20	31
21	38
24	68

4. Find the age of the youngest sailors who is eligible to vote (atleast 18 years old) for each rating level with atleast two such sailors.

OUTPUT:

```
SQL> SELECT rating,MIN(age) FROM sailors
2  WHERE age >= 18
3  GROUP BY rating
4  HAVING COUNT(*) >= 2;
```

RATING	MIN(AGE)
20	31
21	38
24	68

VI. NUMBER ,STRING AND DATE FUNCTIONS

1. Write queries to demonstrate numeric function like:

a) Abs

OUTPUT:

```
SQL> SELECT abs(-45) AS Absolute_vale FROM dual;

ABSOLUTE_VALE
-----
                45
```

b) ceil

OUTPUT:

```
SQL> SELECT ceil(12.8) AS Ceiling_Value FROM dual;

CEILING_VALUE
-----
                13
```

c) Floor

OUTPUT:

```
SQL> SELECT floor(19.7) AS Floor_Value FROM dual;

FLOOR_VALUE
-----
                19
```

d) Trunc

OUTPUT:

```
SQL> SELECT trunc(98.7341) AS Truncated_Value FROM dual;

TRUNCATED_VALUE
-----
                98
```

e) mod

OUTPUT:

```
SQL> SELECT mod(15,2) AS Reminder FROM dual;

REMINDER
-----
                1
```

f) power

OUTPUT:

```
SQL> SELECT power(2,3) AS Result FROM dual;

RESULT
-----
                8
```

g) Sqrt

OUTPUT:

```
SQL> SELECT sqrt(625) AS Root_value FROM dual;

ROOT_VALUE
-----
          25
```

h) round

OUTPUT:

```
SQL> SELECT round(98.8928281,2) AS Rounded_value FROM dual;

ROUNDED_VALUE
-----
          98.89
```

i) log

OUTPUT:

```
SQL> SELECT log(10,1000) AS log_value FROM dual;

LOG_VALUE
-----
          3
```

j) sign

OUTPUT:

```
SQL> SELECT sign(-9000) AS sign_value FROM dual;

SIGN_VALUE
-----
         -1
```

k) least

OUTPUT:

```
SQL> SELECT least(9,66,31,7,91,2) AS smallest_value FROM dual;

SMALLEST_VALUE
-----
                2
```

l) Greatest

OUTPUT:

```
SQL> SELECT greatest(9,66,31,7,91,2) AS largest_value FROM dual;

LARGEST_VALUE
-----
              91
```

2. Write queries to demonstrate string functions

a) lower

OUTPUT:

```
SQL> SELECT lower('VIRAT KOHLI') AS lower_case FROM dual;

LOWER_CASE
-----
virat kohli
```

b) upper

OUTPUT:

```
SQL> SELECT upper('hardik pandey') AS upper_case FROM dual;

UPPER_CASE
-----
HARDIK PANDEY
```


c) ltrim

OUTPUT:

```
SQL> SELECT ltrim('***** MS DHONI #####', '*') AS left_trim_text FROM dual;

LEFT_TRIM_TEXT
-----
MS DHONI #####
```

d) rtrim

OUTPUT:

```
SQL> SELECT rtrim('***** MS DHONI #####', '#') AS right_trim_text FROM dual;

RIGHT_TRIM_TEXT
-----
***** MS DHONI
```

e) length

OUTPUT:

```
SQL> SELECT length('Robinhood') AS text_length FROM dual;

TEXT_LENGTH
-----
          9
```

f) instr

OUTPUT:

```
SQL> SELECT instr('MS DHONI', 'O') AS POSITION FROM dual;

POSITION
-----
        6
```

g) substr

OUTPUT:

```
SQL> SELECT substr('HELLO,WORLD!!!',6,11) AS EXTRACTED FROM dual;

EXTRACTED
-----
,WORLD!!!
```

h) initcap

OUTPUT:

```
SQL> SELECT initcap('welcome,hello world!!') AS Proper_case FROM dual;

PROPER_CASE
-----
Welcome,Hello World!!
```

i) ASCII

OUTPUT:

```
SQL> SELECT ASCII('Z') AS ascii_value FROM dual;

ASCII_VALUE
-----
          90
```

j) chr

OUTPUT:

```
SQL> SELECT chr(90) AS character FROM dual;

C
-
Z
```

3. Write the queries to demonstrate date functions like

a) sysdate

OUTPUT:

```
SQL> SELECT sysdate AS Today FROM dual;

TODAY
-----
20-DEC-25
```

b) Add-months

OUTPUT:

```
SQL> SELECT add_months(sysdate,3) AS after_3_months FROM dual;

AFTER_3_M
-----
20-MAR-26
```

c) Last-day

OUTPUT:

```
SQL> SELECT last_day(sysdate) AS end_of_month FROM dual;

END_OF_MO
-----
31-DEC-25
```

d) Months-between

OUTPUT:

```
SQL> SELECT months_between('20-DEC-2026' , '20-JAN-2026') AS Gap FROM dual;

          GAP
-----
          11
```

e) Next-day

OUTPUT:

```
SQL> SELECT next_day(sysdate , 'FRIDAY') AS NEXT_FRIDAY FROM dual;

NEXT_FRID
-----
26-DEC-25
```

f) trunc

OUTPUT:

```
SQL> SELECT trunc(sysdate) AS Date_without_time FROM dual;

DATE_WITH
-----
20-DEC-25
```

g) extract

OUTPUT:

```
SQL> SELECT
  2  extract(year from sysdate) AS YEAR,
  3  extract(month from sysdate) AS MONTH,
  4  extract(day from sysdate) AS DAY
  5  FROM dual;
```

YEAR	MONTH	DAY
-----	-----	-----
2025	12	20

VII. VIEWS

1. Create a view v1 that contains the name and rating of all sailors.

OUTPUT:

```
SQL> CREATE or REPLACE VIEW v1 AS SELECT sname,rating FROM sailors;
View created.

SQL> SELECT * FROM v1;

SNAME                RATING
-----
RAHUL THOMAS          18
ZAHEER KHAN           13
VIRAT KOHLI           16
ROHIT SHARMA          20
KAPHIL DEV            24
PUSHPA RAJ            21
RAJU DHOLAKPUR        20
TANJIRO KAMADO        25
TOM                   20

9 rows selected.
```

2. Update the view so that rating is incremented by 1 for all who have rating of 24.

OUTPUT:

```
SQL> UPDATE v1 SET rating = rating+1 WHERE rating = 24;
1 row updated.

SQL> SELECT * FROM v1;

SNAME                RATING
-----
RAHUL THOMAS          18
ZAHEER KHAN           13
VIRAT KOHLI           16
ROHIT SHARMA          20
KAPHIL DEV            25
PUSHPA RAJ            21
RAJU DHOLAKPUR        20
TANJIRO KAMADO        25
TOM                   20

9 rows selected.
```

3. Create a non updatable view that contains only the boated and boat name.

OUTPUT:

```
SQL> CREATE or REPLACE VIEW v2 AS SELECT DISTINCT bid,bname FROM boats;
```

```
View created.
```

```
SQL> SELECT * FROM v2;
```

BID	BNAME
200	MARCO
201	RANNI
202	ST.JOSEPH
203	JKD BOATS
204	TAJ
205	STALIN

```
6 rows selected.
```

VIII. MONGODB

a) COMMANDS

1. To create a DB

OUTPUT:

```
test> use myDB;  
switched to db myDB
```

2. To confirm the existence of DB

OUTPUT:

```
myDB> db;  
myDB
```

3. To get a list of all DBs

OUTPUT:

```
myDB> show dbs;  
admin    40.00 KiB  
config   96.00 KiB  
local    96.00 KiB
```

4. To Drop a DB

OUTPUT:

```
myDB> db.dropDatabase();  
{ ok: 1, dropped: 'myDB' }
```

5. To switch to a new DB

OUTPUT:

```
myDB> use myDB1;  
switched to db myDB1
```

6. To display statistics

OUTPUT:

```
myDB1> db.stats();  
{  
  db: 'myDB1',  
  collections: Long('0'),  
  views: Long('0'),  
  objects: Long('0'),  
  avgObjSize: 0,  
  dataSize: 0,  
  storageSize: 0,  
  indexes: Long('0'),  
  indexSize: 0,  
  totalSize: 0,  
  scaleFactor: Long('1'),  
  fsUsedSize: 0,  
  fsTotalSize: 0,  
  ok: 1  
}
```

7. Create a collection and perform various operations.

a) db.createCollection("Person");

OUTPUT:

```
myDB1> db.createCollection("Person");  
{ ok: 1 }  
myDB1> |
```


b) db.student.insert({_id:1, StudName: "Anju", Grade: "VII", Hobbies: "Reading"})

OUTPUT:

```
myDB1> db.student.insert({_id:1,
... StudName: "Anju",
... Grade: "VII",
... Hobbies: "Reading"})
{ acknowledged: true, insertedIds: { '0': 1 } }
```

c) show collections

OUTPUT:

```
myDB1> show collections
Person
student
```

d) db.students.find();

OUTPUT:

```
myDB1> db.student.find();
[
  { _id: 1, StudName: 'Anju', Grade: 'VII', Hobbies: 'Reading' },
  { _id: 3, StudName: 'Aryan David', Grade: 'VII', Hobbies: 'Reading' },
  { _id: 2, StudName: 'Ammu', Grade: 'VII', Hobbies: 'Chess' },
  { _id: 0, StudName: 1, Grade: 1, Hobbies: 'Chess' }
]
myDB1> |
```

e) db.students.find().pretty();

OUTPUT:

```
myDB1> db.student.find().pretty();
[
  { _id: 1, StudName: 'Anju', Grade: 'VII', Hobbies: 'Reading' },
  { _id: 3, StudName: 'Aryan David', Grade: 'VII', Hobbies: 'Reading' },
  { _id: 2, StudName: 'Ammu', Grade: 'VII', Hobbies: 'Chess' },
  { _id: 0, StudName: 1, Grade: 1, Hobbies: 'Chess' }
]
myDB1> |
```

f) db.Students.update({_id:3, StudName:"Aryan David", Grade: "VII"},{\$set:{Hobbies: "Skating"}},{upsert:false});

OUTPUT:

```
myDB1> db.student.updateOne(
... {_id:3, StudName:"Aryan David", Grade: "VII"},
... {$set:{Hobbies: "Skating"}},{upsert:false});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
myDB1> db.student.find();
[
  { _id: 1, StudName: 'Anju', Grade: 'VII', Hobbies: 'Reading' },
  { _id: 3, StudName: 'Aryan David', Grade: 'VII', Hobbies: 'Skating' },
  { _id: 2, StudName: 'Ammu', Grade: 'VII', Hobbies: 'Chess' },
  { _id: 0, StudName: 1, Grade: 1, Hobbies: 'Chess' }
]
```

g) db.Students.update({_id:3, StudName:"Aryan David", Grade: "VII"},{\$set:{Hobbies: "Chess"}},{upsert:true});

OUTPUT:

```
myDB1> db.student.updateOne(
... {_id:3, StudName:"Aryan David", Grade: "VII"},
... {$set:{Hobbies: "Chess"}},{upsert:true});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
myDB1> db.student.find();
[
  { _id: 1, StudName: 'Anju', Grade: 'VII', Hobbies: 'Reading' },
  { _id: 3, StudName: 'Aryan David', Grade: 'VII', Hobbies: 'Chess' },
  { _id: 2, StudName: 'Ammu', Grade: 'VII', Hobbies: 'Chess' },
  { _id: 0, StudName: 1, Grade: 1, Hobbies: 'Chess' }
]
myDB1> |
```

h) db.Students.find({_id:3});

OUTPUT:

```
myDB1> db.student.find({_id:3});
[ { _id: 3, StudName: 'Aryan David', Grade: 'VII', Hobbies: 'Chess' } ]
```

i) db.Students.find({Grade: "VII"});

OUTPUT:

```
myDB1> db.student.find({Grade: "VII"});
[
  { _id: 1, StudName: 'Anju', Grade: 'VII', Hobbies: 'Reading' },
  { _id: 3, StudName: 'Aryan David', Grade: 'VII', Hobbies: 'Chess' },
  { _id: 2, StudName: 'Ammu', Grade: 'VII', Hobbies: 'Chess' }
]
```

j) db.Students.find({StudName:"Ammu"})

OUTPUT:

```
myDB1> db.student.find({StudName:"Ammu"})
[ { _id: 2, StudName: 'Ammu', Grade: 'VII', Hobbies: 'Chess' } ]
myDB1>
```

k) db.Students.find({}, {StudName:1,Grade:1});

OUTPUT:

```
myDB1> db.student.find({}, {StudName:1,Grade:1});
[
  { _id: 1, StudName: 'Anju', Grade: 'VII' },
  { _id: 3, StudName: 'Aryan David', Grade: 'VII' },
  { _id: 2, StudName: 'Ammu', Grade: 'VII' },
  { _id: 0, StudName: 1, Grade: 1 }
]
myDB1> |
```

l) db.Students.find({}, {StudName:1,Grade:1,_id:0});

OUTPUT:

```
myDB1> db.student.find({}, {StudName:1,Grade:1,_id:0});
[
  { StudName: 'Anju', Grade: 'VII' },
  { StudName: 'Aryan David', Grade: 'VII' },
  { StudName: 'Ammu', Grade: 'VII' },
  { StudName: 1, Grade: 1 }
]
myDB1>
```

m) db.Students.find({Grade:"VII"}).pretty().limit(2);

OUTPUT:

```
myDB1> db.student.find({Grade:"VII"}).pretty().limit(2);
[
  { _id: 1, StudName: 'Anju', Grade: 'VII', Hobbies: 'Reading' },
  { _id: 3, StudName: 'Aryan David', Grade: 'VII', Hobbies: 'Chess' }
]
myDB1>
```

n) db.employee.updateMany({}, {\$set: {salary: 50000}})

OUTPUT:

```
myDB1> db.employee.updateMany({}, {$set: {salary: 50000}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
myDB1> db.employee.find();
[
  { _id: 1, empName: 'Goku', dept: 'BBA', salary: 50000 },
  { _id: 2, empName: 'Sonu', dept: 'MBA', salary: 50000 }
]
```

b) MongoDB Queries

1. Create a collection and insert documents into it using insertOne() and insertMany()

- Switch to (or create) a database

OUTPUT:

```
myDB1> use SupplierDB;
switched to db SupplierDB
SupplierDB>
```

- Create a collection named "suppliers" and insert documents
 - Insert one document

OUTPUT:

```
SupplierDB> db.suppliers.insertOne({
...   name: "Geeta",
...   city: "Delhi",
...   phoneno: "99887766",
...   part_name: "P1",
...   color: "red",
...   price: 1500
... });
{
  acknowledged: true,
  insertedId: ObjectId('696e719f6549c00def63b112')
}
SupplierDB> |
```

- Insert many documents

OUTPUT:

```
SupplierDB> db.suppliers.insertMany([
...   {
...     name: "Ravi",
...     city: "Mumbai",
...     phoneno: "88776655",
...     part_name: "P2",
...     color: "green",
...     price: 800
...   },
...   {
...     name: "Anita",
...     city: "Delhi",
...     phoneno: "77665544",
...     part_name: "P3",
...     color: "blue",
...     price: 6000
...   },
...   {
...     name: "Suresh",
...     city: "Chennai",
...     phoneno: "66554433",
...     part_name: "P1",
...     color: "red",
...     price: 100
...   },
...   {
...     name: "Meera",
...     city: "Bangalore",
...     phoneno: "55443322",
...     part_name: "P4",
...     color: "green",
...     price: 450
...   }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('696e71f96549c00def63b113'),
    '1': ObjectId('696e71f96549c00def63b114'),
    '2': ObjectId('696e71f96549c00def63b115'),
    '3': ObjectId('696e71f96549c00def63b116')
  }
}
SupplierDB> |
```

2. Find the count of all suppliers

OUTPUT:

```
SupplierDB> db.suppliers.countDocuments();
5
SupplierDB>
```

3. Select all documents in collection

OUTPUT:

```
SupplierDB> db.suppliers.find({});
[
  {
    _id: ObjectId('696e719f6549c00def63b112'),
    name: 'Geeta',
    city: 'Delhi',
    phoneno: '99887766',
    part_name: 'P1',
    color: 'red',
    price: 1500
  },
  {
    _id: ObjectId('696e71f96549c00def63b113'),
    name: 'Ravi',
    city: 'Mumbai',
    phoneno: '88776655',
    part_name: 'P2',
    color: 'green',
    price: 800
  },
  {
    _id: ObjectId('696e71f96549c00def63b114'),
    name: 'Anita',
    city: 'Delhi',
    phoneno: '77665544',
    part_name: 'P3',
    color: 'blue',
    price: 6000
  },
  {
    _id: ObjectId('696e71f96549c00def63b115'),
    name: 'Suresh',
    city: 'Chennai',
    phoneno: '66554433',
    part_name: 'P1',
    color: 'red',
    price: 100
  },
  {
    _id: ObjectId('696e71f96549c00def63b116'),
```

```
    name: 'Meera',
    city: 'Bangalore',
    phoneno: '55443322',
    part_name: 'P4',
    color: 'green',
    price: 450
  }
]
SupplierDB> |
```

4. Find all records that have city = 'Delhi'

OUTPUT:

```
SupplierDB> db.suppliers.find({ city: "Delhi" });
[
  {
    _id: ObjectId('696e719f6549c00def63b112'),
    name: 'Geeta',
    city: 'Delhi',
    phoneno: '99887766',
    part_name: 'P1',
    color: 'red',
    price: 1500
  },
  {
    _id: ObjectId('696e71f96549c00def63b114'),
    name: 'Anita',
    city: 'Delhi',
    phoneno: '77665544',
    part_name: 'P3',
    color: 'blue',
    price: 6000
  }
]
```

5. Retrieve all documents that have color equal to 'red' or 'green'

OUTPUT:

```
SupplierDB> db.suppliers.find({ color: { $in: ["red", "green"] } });
[
  {
    _id: ObjectId('696e719f6549c00def63b112'),
    name: 'Geeta',
    city: 'Delhi',
    phoneno: '99887766',
    part_name: 'P1',
    color: 'red',
    price: 1500
  },
  {
    _id: ObjectId('696e71f96549c00def63b113'),
    name: 'Ravi',
    city: 'Mumbai',
    phoneno: '88776655',
    part_name: 'P2',
    color: 'green',
    price: 800
  },
  {
    _id: ObjectId('696e71f96549c00def63b115'),
    name: 'Suresh',
    city: 'Chennai',
    phoneno: '66554433',
    part_name: 'P1',
    color: 'red',
    price: 100
  },
  {
    _id: ObjectId('696e71f96549c00def63b116'),
    name: 'Meera',
    city: 'Bangalore',
    phoneno: '55443322',
    part_name: 'P4',
    color: 'green',
    price: 450
  }
]
SupplierDB> |
```

6. Retrieve all documents where part_name is 'P1' or price is less than 200.

OUTPUT:

```
SupplierDB> db.suppliers.find({
...   $or: [
...     { part_name: "P1" },
...     { price: { $lt: 200 } }
...   ]
... });
[
  {
    _id: ObjectId('696e719f6549c00def63b112'),
    name: 'Geeta',
    city: 'Delhi',
    phoneno: '99887766',
    part_name: 'P1',
    color: 'red',
    price: 1500
  },
  {
    _id: ObjectId('696e71f96549c00def63b115'),
    name: 'Suresh',
    city: 'Chennai',
    phoneno: '66554433',
    part_name: 'P1',
    color: 'red',
    price: 100
  }
]
SupplierDB> |
```


7. Update the record of 'Geeta', set city = 'Bombay' and phoneno = '11223344'

OUTPUT:

```
SupplierDB> db.suppliers.updateOne(
...   { name: "Geeta" },
...   { $set: { city: "Bombay", phoneno: "11223344" } }
... );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
SupplierDB> db.suppliers.find().pretty();
[
  {
    _id: ObjectId('696e719f6549c00def63b112'),
    name: 'Geeta',
    city: 'Bombay',
    phoneno: '11223344',
    part_name: 'P1',
    color: 'red',
    price: 1500
  },
]
```

8. Delete all records where price is greater than 5000

OUTPUT:

```
SupplierDB> db.suppliers.deleteMany({ price: { $gt: 5000 } });
{ acknowledged: true, deletedCount: 1 }
SupplierDB> db.suppliers.find();
[
  {
    _id: ObjectId('696e719f6549c00def63b112'),
    name: 'Geeta',
    city: 'Bombay',
    phoneno: '11223344',
    part_name: 'P1',
    color: 'red',
    price: 1500
  },
  {
    _id: ObjectId('696e71f96549c00def63b113'),
    name: 'Ravi',
    city: 'Mumbai',
    phoneno: '88776655',
    part_name: 'P2',
    color: 'green',
    price: 800
  },
  {
    _id: ObjectId('696e71f96549c00def63b115'),
    name: 'Suresh',
    city: 'Chennai',
    phoneno: '66554433',
    part_name: 'P1',
    color: 'red',
    price: 100
  },
]
```

```
  {
    _id: ObjectId('696e71f96549c00def63b116'),
    name: 'Meera',
    city: 'Bangalore',
    phoneno: '55443322',
    part_name: 'P4',
    color: 'green',
    price: 450
  }
]
```

9. Display only the name and city of the supplier

OUTPUT:

```
SupplierDB> db.suppliers.find({}, { _id: 0, name: 1, city: 1 });
[
  { name: 'Geeta', city: 'Bombay' },
  { name: 'Ravi', city: 'Mumbai' },
  { name: 'Suresh', city: 'Chennai' },
  { name: 'Meera', city: 'Bangalore' }
]
SupplierDB> |
```

10. Sort all suppliers on city and display only the first two records.

OUTPUT:

```
SupplierDB> db.suppliers.find({})
...   .sort({ city: 1 })
...   .limit(2);
[
  {
    _id: ObjectId('696e71f96549c00def63b116'),
    name: 'Meera',
    city: 'Bangalore',
    phoneno: '55443322',
    part_name: 'P4',
    color: 'green',
    price: 450
  },
  {
    _id: ObjectId('696e719f6549c00def63b112'),
    name: 'Geeta',
    city: 'Bombay',
    phoneno: '11223344',
    part_name: 'P1',
    color: 'red',
    price: 1500
  }
]
SupplierDB> |
```